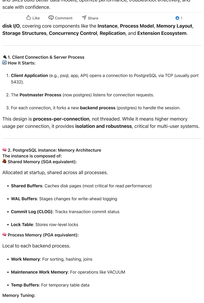

PostgreSQL Complete Architecture: A Deep Dive — PostgreSQL Foundations Series

Wayne C.
Published Oct 23, 2025

+ Follow

PostgreSQL isn't just a relational database — it's a finely tuned, modular system with a robust, extensible design. Understanding its internal architecture helps developers, DBAs, and SREs build better data models, optimize performance, troubleshoot effectively, and scale with confidence.

0 Like | Comment | Share
disk 10

In this article, covering core components like the **Instance**, **Process Model**, **Memory Layout**, **Storage Structures**, **Concurrency Control**, **Replication**, and **Extension Ecosystem**.

🔧 **1. Client Connection & Session Process**
🔥 **How It Starts:**

1. **Client Application** (e.g., psql, app, API) opens a connection to PostgreSQL via TCP (usually port 5432).

2. The **Postmaster Process** (now postgres) listens for connection requests.

3. For each connection, it forks a new **backend process** (postgres) to handle the session.

This design is **process-per-connection**, not threaded. While it means higher memory usage per connection, it provides **isolation and robustness**, critical for multi-user systems.

🧠 **2. PostgreSQL Instance: Memory Architecture**
The instance is composed of:
📦 **Shared Memory (SGA equivalent):**

Allocated at startup, shared across all processes.

- **Shared Buffers**: Caches disk pages (most critical for read performance)
- **WAL Buffers**: Stages changes for write-ahead logging
- **Commit Log (CLOG)**: Tracks transaction commit status
- **Lock Table**: Stores row-level locks

🧮 **Process Memory (PGA equivalent):**

Local to each backend process.

- **Work Memory**: For sorting, hashing, joins
- **Maintenance Work Memory**: For operations like VACUUM
- **Temp Buffers**: For temporary table data

Memory Tuning:

- shared_buffers
- work_mem
- maintenance_work_mem
- effective_cache_size

⚙️ **3. Process Architecture**

PostgreSQL consists of several types of processes:

🔧 **Background Processes:**

- **Postmaster**: Spawns new processes
- **Checkpointer**: Flushes dirty buffers to disk
- **WAL Writer**: Writes WAL changes to log files
- **Background Writer**: Helps maintain buffer pool
- **Autovacuum**: Frees space from dead tuples
- **Stats Collector**: Tracks usage and performance
- **Archiver**: Ships WAL logs for backups/replication

🧩 **Backend Process:**

One per client. Handles SQL parsing, planning, execution, and result delivery.

🔌 **Auxiliary:**

- **Logical Replication Sender/Receiver**
- **WAL Receiver**
- **Replication Launcher**

🧠 **4. SQL Execution Pipeline**

Every SQL query goes through:

1. **Parser** → Converts SQL text into parse tree
2. **Rewriter** → Applies rules (e.g., views, rewrite policies)
3. **Planner/Optimizer** → Chooses best execution path using cost-based analysis
4. **Executor** → Executes plan, accesses tables/indexes, performs joins, filters, etc.

Tools like EXPLAIN ANALYZE expose this path in detail for optimization.

🗂️ **5. Storage Model**

PostgreSQL uses a file-based storage model. Key files:

- **Data files** (one per table/index)
- **Control files** (pg_control, pg_version)
- **WAL files** (pg_wal)
- **Catalog tables** (in pg_catalog schema)
- **Configuration files** (postgresql.conf, pg_hba.conf)

Heap Storage:

- Tables are stored as unsorted heap files
- Each row includes a hidden ctid (tuple ID)
- No clustering unless you use CLUSTER or special indexes

📑 **6. Indexes**

PostgreSQL supports multiple index types:

- **btree**: Default, general-purpose
- **hash**: Fast equality comparisons
- **GIN**: Inverted index, great for arrays & full-text search
- **GIST**: Flexible, supports PostGIS, ranges
- **BRIN**: Ideal for very large, ordered-only tables

You can create expression indexes, partial indexes, and multicolumn indexes.

🔄 **7. Concurrency & MVCC**

PostgreSQL uses **Multiversion Concurrency Control (MVCC)** to support ACID properties without heavy locking.

Each row has hidden columns:

- xmin: inserting transaction ID
- xmax: deleting/updating transaction ID

Readers never block writers. Writers don't block readers. Old versions of rows are retained until VACUUM frees them.

🧹 **8. Vacuum & Disk Management**

To reclaim space from dead rows:

- **VACUUM**: Reclaims space but does not shrink files
- **AUTOVACUUM**: Runs in background, avoids manual tuning
- **ANALYZE**: Collects stats for the optimizer

For aggressive cleanup:

- VACUUM FULL: Rewrites tables
- pg_repack: Shrinks tables/indexes without downtime

📝 **9. WAL (Write-Ahead Logging)**

All changes are first recorded in WAL before hitting data files. This ensures:

- Crash safety
- PITR (Point-in-Time Recovery)
- Replication support

WAL is sequential and append-only, making it fast.

🔁 **10. Replication**
🔄 **Streaming Replication:**

- Asynchronous or synchronous
- Standby reads from WAL sent by primary
- Tools: pg_basebackup, pg_receivewal

🔄 **Logical Replication:**

- Table-based
- Supports bi-directional replication, filtering
- Allows replication across major versions

💾 **11. Backup & Recovery**
Physical Backup:

- pg_basebackup
- File system snapshots (with WAL)

Logical Backup:

- pg_dump, pg_restore

PITR:

- Restore a base backup
- Replay WAL logs up to a target time

🧩 **12. Extensions & Customization**

PostgreSQL is modular. Key extensions:

- PostGIS: GIS/spatial support
- pg_stat_statements: Query insights
- pg_partman: Partition management
- pg_cron: Built-in scheduler
- pgvector: Store and search AI embeddings

You can also write your own C extensions or shared libraries.

🔐 **13. Security Architecture**

- Authentication via pg_hba.conf (md5, scram, cert, LDAP, etc.)
- Roles and privileges (use/group model)
- Row-Level Security (RLS)
- SSL/TLS support
- Extensions for auditing (pgaudit)

📊 **14. Monitoring & Observability**

Core tools:

- pg_stat_activity
- pg_stat_user_tables
- pg_locks
- pg_stat_bgwriter

Popular observability stacks:

- **pgMonitor** (Grafana/Prometheus)
- **pganalyze, pgHero, pg_stat_statements**

🧠 **Summary: Layered PostgreSQL Architecture**

```
Client Apps
  ↓
Postmaster → Backend Processes
  ↓
Parser → Planner → Executor
  ↓
Shared Buffers ← Disk I/O ← WAL
  ↓
Catalogs + Extensions + Replication
```

🧭 **Why Understanding Architecture Matters**

Knowing the internal architecture gives you superpowers:

- Optimize queries and index usage
- Size memory and buffer pools properly
- Architect failover and replication
- Debug deadlocks, bloat, or slow queries

Whether you're deploying PostgreSQL on a laptop, inside Kubernetes, or at global cloud scale — its architecture remains robust, predictable, and extensible.

#postgresql #Foundations

To view or add a comment, sign in

More articles by Wayne C.

Oct 28, 2025
PostgreSQL Environment Variables Setup: Control, Customiz...
PostgreSQL is powerful on its own, but when used effectively with environment variables, it becomes a flexible...

Oct 27, 2025
PostgreSQL Patching: How to Stay Secure, Stable, and Awa...
PostgreSQL is one of the most stable and secure open-source databases available. But no system is immune to bugs,...

Oct 27, 2025
PostgreSQL Major and Minor Version Upgrade: Strategy,...
Upgrading PostgreSQL is not just a version bump — it's a critical operation that affects data integrity, application...

Oct 24, 2025
PostgreSQL Yum Installation: Deploy PostgreSQL on RHEL,...
PostgreSQL is the backbone of many mission-critical systems — but the way you install it can make or break...

Show more

Explore content categories

Career | Productivity | Finance | Soft Skills & Emotional Intelligence | Project Management | Education
Show more

© 2025
Accessibility | Privacy Policy | Copyright Policy | Guest Controls | Language
About | User Agreement | Cookie Policy | Brand Policy | Community Guidelines